Arnold Noah - Logging Process & Audit Subsystem

STC89C52 Microcontroller Multi-Process Simulator - Student Study Guide

Role: Audit Logging Specialist, Storage Engineer & File I/O Architect

1. What is TCP in Simple Terms?

TCP is the **secure pipe that delivers event logs**.

When CoreProcess executes an instruction, it cannot afford to pause and wait for slow hard drives to write.

Instead, it streams a log packet across a local TCP connection on port 5001 to your `LoggerProcess`.

TCP guarantees that:

- Not a single log record is dropped or corrupted in transit.
- Log entries arrive at the disk writer in the exact order the CPU executed them.

2. Is This Logging Setup Beginner-Friendly?

YES. You don't need databases, SQL, or heavy logging frameworks (like Log4j or SLF4J).

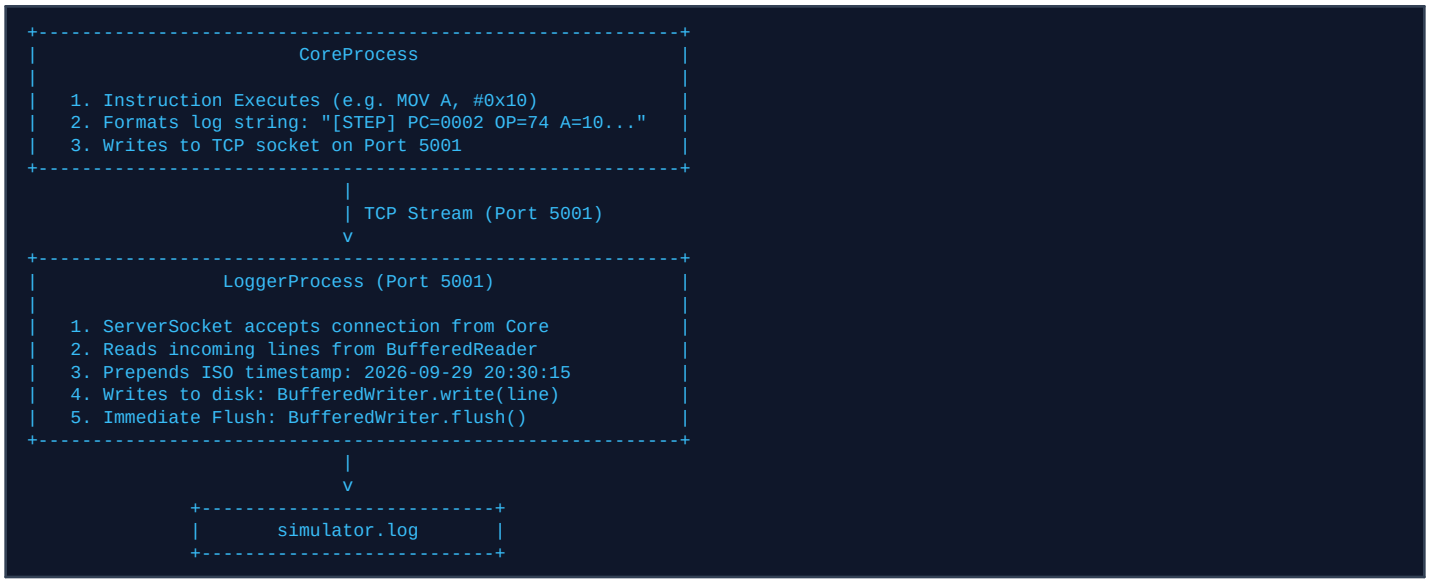
Your entire `LoggerProcess.java` is just **3 simple concepts**:

1. `ServerSocket server = new ServerSocket(5001);` (Listen on port 5001).
2. `String line = reader.readLine();` (Receive log text from CoreProcess).
3. `fileWriter.write(timestamp + " " + line + "\n"); fileWriter.flush();` (Save directly to `simulator.log`).

3. Files Owned & Managed by Arnold

- `src/LoggerProcess.java`: Standalone logging daemon listening on TCP port 5001.
- `simulator.log`: Persistent disk audit trail recording all CPU lifecycle events.

4. Flowchart: Asynchronous Audit Logging Pipeline



5. Structured Log Grammar & Categories

Every line in `simulator.log` adheres to a strict schema:

`[TIMESTAMP] [CATEGORY] [KEY=VALUE ...]`

Log Categories:

1. `[SYS]`: System lifecycle events:

```
2026-09-29 20:30:00 [SYS] Logger started on port 5001
2026-09-29 20:30:05 [SYS] CoreProcess connected
```

2. `[EXEC]`: Individual instruction execution cycles:

```
2026-09-29 20:30:06 [EXEC] [CPU] PC=0002|A=10|CY=false
```

3. `[QUEUE]`: Circular queue operations:

```
2026-09-29 20:30:07 [QUEUE] OP=ENQUEUE VAL=10 SIZE=1
2026-09-29 20:30:08 [QUEUE] OP=DEQUEUE VAL=10 SIZE=0
```

4. `[MEM]`: Memory writes and updates:

```
2026-09-29 20:30:09 [MEM] ADDR=0030 DATA=FF
```

5. `[ERROR]`: Hardware faults or abnormal terminations:

```
2026-09-29 20:30:10 [ERROR] STACK_OVERFLOW SP=0F
```

---

6. Why a Dedicated Logging Process?

- **Crash Resilience:** If `CoreProcess` experiences a fatal division-by-zero or memory fault, the `LoggerProcess` remains alive and retains the complete post-mortem audit trail up to the exact instruction that crashed.
- **Zero Disk Latency Impact on CPU:** Disk I/O is hundreds of times slower than memory operations. Separating logging into a separate process prevents disk write spikes from freezing microcontroller cycle timing.

---

7. How Arnold Can Add Features as College Teaches More

- **Adding a New Log Level (e.g. `[WARN]` or `[PERF]`):**

In `LoggerProcess.java`, customize the format string to include execution cycle timestamps or memory usage metrics.

- **Auto-Archiving Old Logs:**

Add a simple check: if `file.length() > 1_000_000` (1MB), rename `simulator.log` to `simulator_old.log` and start a fresh file.

---

8. Arnold's 5 Hostile Examiner Questions & Defenses

Q1: "Why do you call `.flush()` after every single log write?"

Answer: *"Standard operating system file writers buffer data in RAM before writing to physical disk. If the computer loses power or the process terminates unexpectedly, un-flushed data in the buffer is permanently lost. Calling `.flush()` forces the OS to immediately commit every log transaction to physical storage."*

Q2: "What happens if LoggerProcess is started after CoreProcess?"

Answer: *"In `CoreProcess.java`, socket connection to port 5001 is enclosed in a fail-safe try-catch block. If LoggerProcess is not yet online, CoreProcess logs a console alert and continues executing without crashing. Once LoggerProcess comes online, logs can resume."*

Q3: "What port does your logging server listen on?"

Answer: *"LoggerProcess binds to TCP port 5001 on localhost (`127.0.0.1`). CoreProcess connects as a client to stream events."*

Q4: "Show me where the log file is stored and how old logs are managed."

Answer: *"Logs are written to `simulator.log` in the root working directory. The file is opened in append mode (`new FileWriter('simulator.log', true)`), preserving historical execution traces across consecutive simulation runs."*

Q5: "How does your logger guarantee timestamps are accurate?"

Answer: *"LoggerProcess generates timestamps at the moment the log packet arrives using* `java.time.LocalDateTime` *formatted to* `yyyy-MM-dd HH:mm:ss.SSS`*. This ensures monotonic time tracking for all execution events."*